

CL-AWS2 — AWS avancés + serverless

Jour 3 — Orchestration et événements : Step Functions, EventBridge

Assembler les briques des jours 1 et 2 en une architecture — celle du TP3

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- **Lire et écrire** une machine d'état Step Functions : Task, Choice, Wait, Parallel, Map, Retry/Catch.
- **Choisir** entre workflow standard et express, et citer les cas d'usage admin (pipelines, remédiation).
- **Créer** une règle EventBridge : pattern d'événement, cible — et **remplacer un cron** par un événement planifié.
- **Décider** honnêtement : serverless ou EC2/conteneurs, selon coût, latence, durée, verrouillage.
- **Dessiner** l'architecture StockLine serverless complète — celle du TP3.
- **Appliquer** les 3 règles de sécurité serverless : un rôle par fonction, moindre privilège, pas de secrets en dur.

Fin de journée : quiz de synthèse des **3 jours** serverless.

Plan de la journée

1. **Step Functions** — machines d'état, états, standard vs express, cas d'usage admin.
2. **EventBridge** — bus, règles, patterns, cibles, événements planifiés.
3. **Architecture événementielle** — quand serverless, quand pas ; StockLine serverless complète.
4. **Sécurité serverless** — en 3 slides.

Démo : la machine d'état du pipeline d'inventaire ( 3-1). Exercices : prédire le comportement d'une machine d'état ( 3-1), serverless ou pas ( 3-2). Quiz de synthèse : 10 questions sur les 3 jours.

1. Step Functions

Quand une Lambda ne suffit plus : orchestrer

Le problème : le « Lambda spaghetti »

Hier, notre ingestion faisait **tout dans une fonction** : valider, écrire, notifier. Ça tient... tant que c'est simple. Ajoutez : « archiver le fichier, réessayer 3 fois l'écriture, prévenir un humain si la validation échoue, traiter 50 fichiers en parallèle » :

- Une **méga-Lambda** ? Elle frôle le timeout, mélange les responsabilités, et un échec au milieu laisse un état à moitié fait.

- Des **Lambda qui s'appellent entre elles** ? Qui réessaie ? Qui sait où en est le fichier n°37 ? Où est la vue d'ensemble ? C'est le *spaghetti* : illisible, indéboguable.

Step Functions sort la logique d'enchaînement du code : le workflow devient une **machine d'état** déclarée en JSON, exécutée, tracée et **dessinée** par AWS.

Pour l'admin : le pipeline devient un **document versionnable** + une console où chaque exécution montre, en vert et rouge, où elle en est. Fini le « je greppe les logs de 4 fonctions ».

Machine d'état : les concepts

Une **machine d'état** (*state machine*) est décrite en **ASL** (*Amazon States Language*, du JSON) :

- des **états** (*states*) : chacun fait une chose (invoker une Lambda, décider, attendre...) ;
- des **transitions** : **Next** désigne l'état suivant, **End: true** termine ;
- un **document JSON** circule d'état en état : la sortie de l'un devient l'entrée du suivant.

```
{
  "Comment": "Squelette minimal",
  "StartAt": "PremierEtat",
  "States": {
    "PremierEtat": { "Type": "Pass", "Next": "Fin" },
    "Fin":          { "Type": "Succeed" }
  }
}
```

Chaque **exécution** (*execution*) est indépendante, identifiée, rejouable, et son historique complet est conservé (90 jours en standard).

L'état Task : faire travailler un service

Task délègue le travail à un service — le plus souvent une Lambda :

```
"IngererLeProduit": {
  "Type": "Task",
  "Resource": "arn:aws:lambda:eu-west-3:123456789012:function:abc-ingestion",
  "TimeoutSeconds": 60,
  "Retry": [
    {
      "ErrorEquals": ["Lambda.ServiceException",
        "Lambda.TooManyRequestsException"],
      "IntervalSeconds": 2,
      "MaxAttempts": 3,
      "BackoffRate": 2.0
    }
  ],
  "Catch": [
    { "ErrorEquals": ["States.ALL"], "Next": "PrevenirUnHumain" }
  ]
}
```

```
],
"Next": "EtapeSuivante"
}
```

Retry/Catch déclaratifs : les 3 tentatives espacées de 2 s, 4 s, 8 s ne sont plus du code Python à écrire — c'est la moitié de la valeur de Step Functions. Il existe aussi des intégrations directes sans Lambda (`sns:publish`, `DynamoDB...`).

L'état Choice : l'aiguillage

Choice route selon le contenu du document JSON — le `if/elif/else` du workflow :

```
"FichierExploitable": {
  "Type": "Choice",
  "Choices": [
    {
      "Variable": "$.lignes",
      "NumericGreaterThan": 0,
      "Next": "TraiterChaqueProduit"
    },
    {
      "Variable": "$.format",
      "StringEquals": "inconnu",
      "Next": "PrevenirUnHumain"
    }
  ],
  "Default": "FichierVide"
}
```

- `$.lignes` : un **chemin JSONPath** dans le document qui circule.
- Les **Choices** sont évalués **dans l'ordre** ; **Default** attrape le reste — sans lui, une donnée imprévue fait échouer l'exécution (`States.NoChoiceMatched`).

Wait et Parallel

Wait — suspendre l'exécution, sans facturer de calcul (personne ne « dort » dans une Lambda) :

```
"LaisserLIndexSePropager": { "Type": "Wait", "Seconds": 5, "Next": "Suite" }
```

Variantes : **Timestamp** (jusqu'à une date), **SecondsPath** (durée lue dans le document). En standard, un Wait peut durer **des jours** — impossible dans une Lambda limitée à 15 min.

Parallel — plusieurs branches en même temps, on attend que **toutes** finissent :

```

"FinaliserEnParallele": {
  "Type": "Parallel",
  "Branches": [
    { "StartAt": "NotifierLEquipe", "States": { "NotifierLEquipe": { "Type":
"Pass", "End": true } } },
    { "StartAt": "ArchiverLeFichier", "States": { "ArchiverLeFichier": { "Type":
"Pass", "End": true } } }
  ],
  "Next": "TraitementReussi"
}

```

Si **une** branche échoue, le Parallel échoue (sauf Catch). Résultat : un **tableau** avec la sortie de chaque branche.

Map : la boucle sur une collection

Map exécute un sous-workflow **pour chaque élément d'un tableau** :

```

"TraiterChaqueProduit": {
  "Type": "Map",
  "ItemsPath": "$.produits",
  "MaxConcurrency": 2,
  "ItemProcessor": {
    "StartAt": "IngererLeProduit",
    "States": { "IngererLeProduit": { "Type": "Pass", "End": true } }
  },
  "ResultPath": "$.resultats",
  "Next": "Suite"
}

```

- **ItemsPath** : le tableau d'entrée ; chaque élément devient l'entrée d'une itération.
- **MaxConcurrency** : le parallélisme (0 = illimité ; 2 = deux à la fois — pour ménager une API en aval).
- Les autres types d'états, pour mémoire : **Pass** (transformer/tester), **Succeed**, **Fail** (fin contrôlée avec **Error/Cause**).



L'exercice 3-1 vous fera **prédire** le comportement d'une machine complète avant de l'exécuter.

Standard vs Express

	Standard (notre choix)	Express
Durée max d'une exécution	1 an	5 minutes
Facturation	à la transition d'état (~25 €/M)	à la durée × mémoire (comme Lambda)

	Standard (notre choix)	Express
Historique	complet, console, 90 j	via CloudWatch Logs
Exécution	exactement une fois	au moins une fois (doublons possibles)
Débit	milliers/s	> 100 000/s
Cas type	pipelines, remédiation, approbations humaines	traitement d'événements à très haut volume

💰 Free-tier permanent standard : **4 000 transitions/mois** — nos démos en consomment quelques dizaines.

Règle simple : **standard par défaut** ; express quand le volume est énorme **et** le workflow court **et** idempotent.

Cas d'usage pour l'admin cloud

Step Functions n'est pas réservé aux développeurs — deux familles de cas très « métier admin » :

Pipelines de traitement (notre fil rouge) : valider → ingérer (Map) → attendre → notifier + archiver (Parallel).
Chaque exécution = un fichier, visible, rejouable.

Remédiation automatique :

```
[Alarme CloudWatch] -> [machine d'état]
  Task: diagnostiquer (Lambda decrit l'instance)
  Choice: disque plein ? -> Task: purger les logs -> Wait 2 min
                        -> Choice: résolu ? -> Succeed
  Default: Task: créer un ticket + notifier l'astreinte -> Fail
```

Autres classiques : rotation de secrets orchestrée, snapshots multi-comptes, onboarding d'un nouvel arrivant (créer IAM + envoyer les accès + attendre validation manager).

Le fil conducteur du cursus, encore : **automatiser, superviser, sécuriser** — ici, l'automatisation devient un document que l'on peut auditer.



Démo 3-1

La machine d'état du pipeline d'inventaire

[demos/08-cl-aws2/demo-3-1-step-functions.md](#)

[state-machine-inventaire.json](#) : Choice, Map, Wait, Parallel — exécutée, visualisée, cassée exprès



Exercice 3-1 — Lire une machine d'état

exercices/08-cl-aws2/exercice-3-1-lire-machine-etat.md — 40 min

On vous donne une machine d'état de **remédiation d'espace disque** (JSON complet, 8 états) et 4 entrées différentes.

Pour chaque entrée, **sans l'exécuter** :

1. Listez la séquence exacte des états traversés.
2. Donnez le résultat final (Succeed ou Fail, et le document de sortie).
3. Comptez les transitions facturées.

Puis deux questions de modification : « où ajouter un Retry pour tolérer une Lambda throttlée ? », « que se passe-t-il si on supprime le Default du Choice ? ».

C'est exactement la compétence attendue en exploitation : **prédire** un workflow avant de le lancer sur la prod.

2. EventBridge

Le bus d'événements — et la retraite de cron

EventBridge : le bus d'événements

EventBridge est un **bus** : des sources y publient des événements, des **règles** filtrent, et routent vers des **cibles**.

SOURCES	BUS	CIBLES
services AWS ----->	+-----+ règle 1 -->	Lambda
(EC2, S3, ...)	default bus	règle 2 -->
vos applis ----->	+-----+ règle 3 -->	SQS, SNS, ...
planificateur ----->	filtre par PATTERN	

Différence clé avec SNS : on ne s'abonne pas à un topic choisi par l'émetteur — on écrit une règle qui **filtre sur le contenu** de l'événement.

Le **bus par défaut** reçoit déjà, gratuitement, les événements de votre compte : changement d'état EC2, résultat d'un teardown, findings de sécurité... Vous avez déjà un flux d'événements — il suffit d'y brancher des règles. 💰 Les événements AWS et planifiés sont gratuits ; événements custom ~1 €/M.

Règles et patterns d'événements

Un événement est un JSON normalisé ; un **pattern** est un JSON qui décrit ce qu'on veut attraper — champ présent = doit correspondre, valeurs en **tableau** = « ou » :

```
{
  "version": "0", "source": "aws.ec2", "detail-type": "EC2 Instance State-change
```

```
Notification",
  "region": "eu-west-3",
  "detail": { "instance-id": "i-0abc123", "state": "stopped" }
}
```

```
{
  "source": ["aws.ec2"],
  "detail-type": ["EC2 Instance State-change Notification"],
  "detail": { "state": ["stopped", "terminated"] }
}
```

Cas admin immédiat : cette règle + une cible SNS = « **préviens-moi quand une instance s'arrête** », sans une ligne de code. Préfixes (`{ "prefix": "abc-" }`), négations (`anything-but`), numériques : le filtrage est riche.

Les cibles — et l'événement planifié qui remplace cron

Une règle route vers une ou plusieurs **cibles** : Lambda, Step Functions, SQS, SNS, ECS...

Le cas que tout admin utilise dès la première semaine : **le planning**. Souvenez-vous du TP1 — la sauvegarde de StockLine tournait via `cron` sur la VM. Mais qui surveille cron ? Que devient-il quand la VM disparaît ?

```
aws scheduler create-schedule \
  --name $PREFIX-rapport-stock-quotidien \
  --schedule-expression "cron(0 7 * * ? *)" \
  --schedule-expression-timezone "Europe/Paris" \
  --flexible-time-window Mode=OFF \
  --target '{"Arn":"arn:aws:lambda:eu-west-3:ACCOUNT_ID:function:abc-produits",
            "RoleArn":"arn:aws:iam::ACCOUNT_ID:role/abc-scheduler-role"}' \
  --region eu-west-3
```

Même syntaxe cron (à un champ près : `?`), mais : **pas de machine**, retries intégrés, fuseau horaire géré, historique des invocations, et le rôle IAM dit précisément ce que le planificateur a le droit de lancer.

SQS vs SNS vs EventBridge : le tableau de la maturité

La question piège d'hier (exercice 2-2) trouve sa réponse :

Besoin	Service
Absorber du travail, garantir le traitement, lisser un pic	SQS
Diffuser un message à des abonnés connus (dont emails/SMS)	SNS
Router des événements selon leur contenu , réagir aux événements AWS , planifier	EventBridge

Besoin	Service
Diffuser puis absorber	SNS → SQS (fan-out)
Router puis absorber	EventBridge → SQS

Repères rapides : besoin d'emails → SNS ; besoin de filtrage riche par contenu ou d'événements de services AWS → EventBridge ; besoin que « le travail soit fait, même en panne » → il y a une file SQS quelque part.

Honnêteté : SNS et EventBridge se recouvrent partiellement — dans un système neuf orienté événements, EventBridge est souvent le meilleur défaut ; SNS reste imbattable pour la notification humaine et le très haut débit.

3. Architecture événementielle

Assembler — et savoir quand ne PAS faire de serverless

Penser en événements

Le fil des 3 jours, dit autrement : on est passé de « des serveurs qui attendent » à « du code qui **réagit** » :

- *un client appelle* → API Gateway → Lambda (J1)
- *un fichier arrive* → S3 → Lambda → DynamoDB → SNS (J2)
- *un traitement se complique* → Step Functions l'orchestre (J3)
- *il est 7 h / une instance s'arrête* → EventBridge → cible (J3)

Propriétés de ce style d'architecture :

- **Découplage** : chaque brique ignore qui la précède et la suit — on ajoute un consommateur sans toucher au producteur.
- **Élasticité** : chaque maillon scale indépendamment.
- **Asynchrone par défaut** : la supervision change — on ne surveille plus des processus, mais des **files (profondeur), des DLQ, des taux d'erreur et des durées**.

Quand le serverless brille

- **Trafic irrégulier ou faible** : API internes, back-offices, outils d'équipe — le cas StockLine. 0 requête = 0 €.
- **Traitements événementiels** : fichiers déposés, webhooks, notifications, glue entre services.
- **Tâches planifiées** : rapports, sauvegardes, nettoyages — tout ce qui vivait dans cron.
- **Pics extrêmes et imprévisibles** : 0 → 1 000 exécutions parallèles sans préavis, sans ASG à pré-chauffer.
- **Petites équipes** : personne à réveiller pour patcher un OS ; l'exploitation se concentre sur les permissions et la supervision.

Et le coût de possession complet : pas d'AMI à maintenir, pas de capacité à planifier, haute disponibilité multi-AZ incluse — des heures d'admin économisées qui ne figurent sur aucune facture AWS.

Quand le serverless est le mauvais choix

Dire non fait partie du métier — les vraies limites :

- **Durée** : 15 min max par exécution Lambda. Un batch de 2 h, un rendu vidéo, une migration → conteneurs/EC2 (ou découpage Step Functions, si ça se découpe).
- **Latence stricte** : cold starts de 100 ms à 2 s. Trading, temps réel dur, SLA < 50 ms constants → non.
- **Charge élevée et CONSTANTE** : à 100 % d'utilisation 24 h/24, l'EC2 réservée ou le conteneur Fargate devient **moins cher** que des millions de Go-s Lambda. Le serverless facture l'usage — tant mieux si l'usage est creux, tant pis s'il est plein.
- **Verrouillage (lock-in)** : event format API Gateway, ASL, modèle DynamoDB — migrer ailleurs se paie. Un conteneur, lui, tourne partout.
- **Dépendances lourdes / états longs** : WebSockets persistants, GPU, licences liées à une machine.

Aucune honte à conclure « EC2/conteneurs » : le TP2 n'était pas une erreur, c'était un autre point du spectre. Demain (J4), justement : les conteneurs sur AWS.

Le tableau de décision honnête

Critère	Lambda (serverless)	Conteneurs (ECS/Fargate — J4)	EC2 (TP2)
Coût à trafic faible/irrégulier	✓ quasi nul	moyen (tâche allumée)	✗ plein tarif 24/7
Coût à charge forte et constante	✗ cher	✓ bon	✓ le meilleur (réservé)
Latence première requête	✗ cold start	✓ stable	✓ stable
Durée max d'un traitement	✗ 15 min	✓ illimitée	✓ illimitée
Effort d'exploitation (OS, patch, scaling)	✓ minimal	moyen	✗ maximal
Portabilité / verrouillage	✗ fort	✓ image standard	moyen
Mise à l'échelle	✓ instantanée, par requête	bonne (minutes)	lente (ASG)

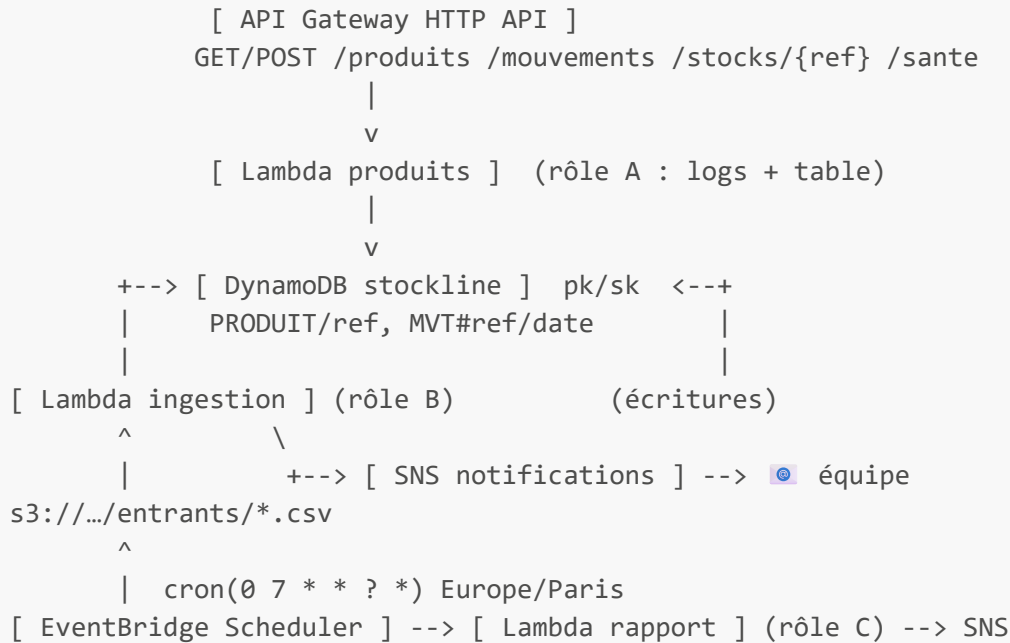
Questions à poser, dans l'ordre : *combien de temps dure un traitement ? le trafic est-il constant ? quelle latence est contractuelle ? qui exploite ?* — le service se déduit, pas l'inverse.

StockLine serverless complète — l'architecture du TP3

```

navigateur (front statique sur S3)
|
v  HTTPS + CORS

```



Chaque flèche = une **permission IAM explicite**. Coût à vide : **0 €**.

TP2 → TP3 : la table de correspondance

Brique du TP2	Équivalent TP3	Ce qui change pour vous
ALB	API Gateway HTTP API	plus d'instance, CORS natif, facturé/requête
EC2 + ASG (FastAPI)	Lambda produits	plus d'OS ; un handler, un rôle, un zip
RDS PostgreSQL	DynamoDB	plus de SQL ; clés conçues d'avance
cron sur la VM	EventBridge Scheduler	plus de machine ; retries et fuseau gérés
S3 front statique	S3 front statique	inchangé !
scripts boto3 d'exploitation	scripts boto3	inchangé — vos acquis restent

Ce tableau **est** le plan du TP3 J1 (noté) : la même API, les mêmes endpoints, le contrat de **GET /stocks/{ref}** identique — l'infrastructure en dessous, elle, a changé de siècle.

Exercice 3-2 — Serverless ou pas ?

[exercices/08-cl-aws2/exercice-3-2-serverless-ou-pas.md](#) — 40 min

Cinq systèmes réels à arbitrer, budget et SLA fournis :

1. API de badge d'accès des bureaux (400 requêtes/jour, 3 sites).
2. Encodage vidéo des formations (fichiers de 2 Go, 40 min de traitement).
3. Moteur de recherche produit (800 requêtes/s **constantes**, latence p99 < 80 ms).
4. Chaîne de traitement des notes de frais (pics le 30 du mois, zéro perte tolérée).
5. Site vitrine + formulaire de contact.

Livrable : pour chacun, **le choix** (Lambda / conteneurs / EC2 / mixte), **deux arguments chiffrés** tirés du tableau de décision, et **le risque principal** du choix retenu. Débrief en binômes : défendez un choix différent du vôtre.

4. Sécurité serverless

Trois slides, trois réflexes

Réflexe 1 — Un rôle par fonction

Cette semaine, nous avons créé **trois rôles distincts** — ce n'était pas du zèle :

Fonction	Rôle	Permissions
produits	<code>\$PREFIX-lambda-produits-role</code>	logs + Get/Put/Query sur la table
ingestion	<code>\$PREFIX-lambda-ingestion-role</code>	logs + GetObject sur entrants/ + BatchWrite + Publish sur le topic
rapport (TP3)	rôle dédié	logs + Query + Publish

L'anti-pattern à bannir : **un rôle « lambda-role » partagé** qui accumule les permissions de tout le monde. Une seule fonction compromise (dépendance piégée, injection) = tout le périmètre du rôle est compromis.

Un rôle par fonction = le **rayon d'explosion** (*blast radius*) d'une compromission se limite à ce que **cette** fonction devait faire.

Réflexe 2 — Moindre privilège, ressource par ressource

Relisez nos politiques : jamais `"Action": "dynamodb:*"`, jamais `"Resource": "*" :`

```
{ "Action": ["dynamodb:GetItem", "dynamodb:PutItem", "dynamodb:Query"],  
  "Resource": "arn:aws:dynamodb:eu-west-3:ACCOUNT_ID:table/abc-stockline" }
```

- La fonction produits n'a **pas** `DeleteItem` : elle n'en a pas besoin, donc elle ne l'a pas. Ni `Scan`, d'ailleurs — la politique **interdit physiquement** l'anti-pattern d'hier.
- L'ingestion lit `entrants/*` — pas tout le bucket : elle ne peut pas relire ses propres archives ni exfiltrer autre chose.
- Méthode au TP3 : écrire le code d'abord, lister les appels boto3, traduire **chaque appel** en une ligne de politique. Pas un de plus. L'erreur `AccessDenied` est votre amie : elle nomme exactement l'action manquante.

Réflexe 3 — Ni secrets en dur, ni confiance dans les événements

Secrets :

- Jamais dans le code ni dans le zip (le zip se télécharge avec `lambda:GetFunction...`).
- Les variables d'environnement : acceptables pour de la **configuration** (nom de table, ARN de topic), pas pour un mot de passe — lisibles par quiconque voit la configuration.
- La bonne maison des secrets : **SSM Parameter Store** (SecureString) ou **Secrets Manager**, lus au cold start avec une permission dédiée. Approfondi en CL-SECU.
- Notez ce que DynamoDB nous a offert ici : **pas de mot de passe du tout** — l'authentification, c'est le rôle IAM.

Événements :

- Un event S3, SQS ou API Gateway est une **entrée non fiable** : notre `lambda_ingestion` valide chaque ligne du CSV avant d'écrire. Gardez ce réflexe au TP3 — c'est un critère de la grille.

Récap — les 3 jours serverless en une slide

- **J1 — Lambda + API Gateway** : `handler(event, context)`, rôle d'exécution, mémoire=CPU, timeout 15 min max, cold start, logs CloudWatch (**REPORT**), zip, HTTP API + routes + CORS. La même API que le TP2, 0 € à vide.
- **J2 — Données et découplage** : DynamoDB conçu par les schémas d'accès (query, jamais scan), événements S3 (asynchrone, unquote_plus, préfixes disjoints), SQS (absorber ; visibilité, DLQ, idempotence), SNS (diffuser ; fan-out).
- **J3 — Orchestration** : Step Functions (Task/Choice/Wait/Parallel/Map, Retry/Catch déclaratifs, standard vs express), EventBridge (patterns, cibles, cron sans machine), tableau de décision serverless vs conteneurs vs EC2, un rôle par fonction + moindre privilège.

TP3 J1 (noté) : vous reconstruirez StockLine serverless seuls — tout ce qu'il faut est dans ces trois jours, le mini-TP et la cheatsheet `cheatsheet/08-cl-aws2-j1-j3.md`.



Quiz de synthèse — 3 jours serverless

10 questions — 20 minutes

Réponses commentées demain matin

Quiz — questions 1 et 2

Question 1 — Classez ces trois traitements dans la bonne case (Lambda / Step Functions + Lambda / conteneur) en justifiant par une limite chiffrée : (a) redimensionner une image à l'upload, (b) un batch comptable de 3 h non découppable, (c) un pipeline valider → écrire → notifier avec 3 tentatives par étape.

Question 2 — Dans une machine d'état, quelle est la différence entre **Retry** et **Catch** sur un état Task ? Donnez un cas d'usage de chacun dans notre pipeline d'inventaire.

Quiz — questions 3 et 4

Question 3 — Une exécution Step Functions **standard** traite un tableau de 4 produits via un état Map (**MaxConcurrency**: 2). Expliquez ce que fait **MaxConcurrency**, et pourquoi on limiterait le parallélisme alors que « le serverless scale tout seul ».

Question 4 — Votre workflow doit attendre 48 h une validation humaine avant de continuer. Standard ou express ? Deux raisons. Et pourquoi ce « Wait » de 48 h ne coûte-t-il presque rien, là où une EC2 qui attend coûterait 48 h de machine ?

Quiz — questions 5 et 6

Question 5 — Écrivez (ou décrivez champ par champ) le pattern EventBridge qui attrape les événements « une instance EC2 dont l'identifiant commence par **i-0** passe à l'état **stopped** ou **terminated** », et citez deux cibles pertinentes pour un admin.

Question 6 — Votre sauvegarde StockLine tournait en cron sur la VM du TP1. Donnez trois avantages concrets d'un schedule EventBridge sur ce cron — et un inconvénient honnête.

Quiz — questions 7 et 8

Question 7 — « On met tout en serverless, c'est toujours moins cher. » Contre-argumentez avec les deux situations vues en cours où Lambda devient le **mauvais** choix économique ou technique, chiffres à l'appui.

Question 8 — Dans l'architecture StockLine serverless du TP3, listez les trois rôles IAM et, pour chacun, les permissions exactes qu'il porte. Pourquoi ne pas utiliser un seul rôle pour les trois fonctions ?

Quiz — questions 9 et 10

Question 9 — Remettez ces événements dans l'ordre chronologique pour un dépôt de CSV dans le pipeline complet, en précisant quel service parle à quel service et en mode synchrone ou asynchrone : *notification SNS envoyée* — *put_item DynamoDB* — *événement S3 émis* — *invocation Lambda* — *aws s3 cp* — *confirmation d'abonnement (piège)*.

Question 10 — Votre collègue stocke le mot de passe d'une API externe dans une variable d'environnement Lambda et vous dit « c'est chiffré, c'est bon ». Que lui répondez-vous, en trois points : le risque réel, la bonne solution AWS, et l'exemple de cette semaine où le problème du mot de passe a disparu par conception.

À demain

Jour 4 — Les conteneurs arrivent sur AWS

Le tableau de décision d'aujourd'hui avait une colonne du milieu : **les conteneurs** — durée illimitée, portabilité, coût stable.

Demain : **ECR** (le registre d'images) et **ECS/Fargate** (faire tourner des conteneurs... sans gérer les serveurs non plus) — l'avant-goût du bloc Docker/Kubernetes. Puis J5 : **CloudFormation**, pour décrire toute l'infra de cette semaine dans un fichier. Et J6 : l'art de faire baisser la facture. 🙌